Information Retrieval Final Assignment Report

Yutao Liu(s4213211) Kerem Cimilli(s4464346)

May 28, 2025

Abstract

In this report, we investigate the effectiveness of cross-encoder re-rankers and query expansion using large language models (LLMs) in neural information retrieval. We fine-tune three cross-encoder models, apply ensemble methods for result fusion, and evaluate their performance. Additionally, we explore prompt-based query expansion with and without pseudo-relevance feedback. Our findings highlight the strengths of model fusion and LLM-driven reformulations in improving retrieval quality.

1 Introduction

In recent years, re-ranking methods using transformer-based cross-encoders have become central to achieving state-of-the-art performance in information retrieval tasks. These models evaluate the relevance of query-passage pairs with high accuracy but require careful fine-tuning and evaluation across large datasets.

To further improve retrieval effectiveness, ensemble methods have been proposed to fuse the outputs of multiple re-rankers. The `ranx.fuse` library provides a practical framework for implementing and evaluating such fusion strategies, including Reciprocal Rank Fusion (RRF), CombSUM, and other classic and learning-based methods [1].

In addition, the emergence of large language models (LLMs) has opened new possibilities for semantic query reformulation. Query expansion via LLM prompting has shown promising results in enhancing retrieval coverage without requiring manual query engineering [2].

This report presents a systematic study of three cross-encoder models fine-tuned on MSMARCO data, their individual and fused performances on the TREC DL'19 benchmark, and optional experiments on query expansion with LLMs. Our goal is to compare both model-level and ensemble-level effectiveness using robust evaluation metrics.

2 Task 1: Fine-Tuning and Evaluation of Cross-Encoders

2.1 Experimental Setup

We fine-tuned three different cross-encoder models using the MSMARCO dataset for a duration of one hour each, as specified in the assignment. Each model was fine-tuned for approximately 1 hour using binary classification with positive and negative passages. We recorded the number of training steps reached within this time limit.

2.2 Evaluation Results

After training, we evaluated each fine-tuned model on the TREC DL'19 benchmark dataset (43 queries) using the following metrics: NDCG@10, Recall@100, and MAP@1000.

2.3 Discussion

Among the three models, `cross-encoder/ms-marco-TinyBERT-L-2-v2` achieved the highest performance across all three evaluation metrics. Despite being a lightweight model, TinyBERT outperformed both

Model	Steps	NDCG@10	Recall@100	MAP@1000
MiniLM	65000	67.03	49.25	42.88
TinyBERT	52000	**69.19**	**50.35**	**45.36**
DistilRoBERTa	58000	65.63	47.27	42.09

Table 1: Evaluation results of the three fine-tuned cross-encoders on TREC DL'19 queries.

MiniLM and DistilRoBERTa, possibly due to faster convergence or better suitability for the MSMARCO fine-tuning setup.

However, a higher score on this benchmark does not guarantee better generalization across all unseen queries. Each model may respond differently to varying query types, document lengths, or semantic complexity. Additionally, the one-hour training constraint may not have allowed the models to reach full convergence.

Potential limitations of these models include: Maximum input length limited, no use of contextual or multi-hop retrieval, limiting deeper reasoning or training was cut off manually at a fixed time.

In conclusion, TinyBERT showed the most promising results for this specific reranking task, though future work could explore longer training, query expansion, or ensemble methods to further boost effectiveness.

3 Task 2: Evaluation of Ensemble Fusion Methods

3.1 Setup

We evaluated five ensemble methods for combining the ranking outputs of the three cross-encoder models trained in Task 1: MiniLM, TinyBERT, and DistilRoBERTa. All fusion results were evaluated on the same TREC DL'19 query set (43 queries), using the metrics NDCG@10, Recall@100, and MAP@1000.

3.2 Results

Fusion Method	NDCG@10	Recall@100	MAP@1000
RRF	**0.7092**	0.5075	**0.4629**
CombSUM	0.7061	**0.5103**	0.4608
CombMNZ	0.7061	**0.5103**	0.4608
BordaFuse	0.7056	0.5069	0.4621
LogISR	0.6801	0.5021	0.4487

Table 2: Performance of ensemble methods across three evaluation metrics.

3.3 Discussion

As shown in Table 2, the RRF method achieved the highest scores for both NDCG@10 and MAP@1000, while CombSUM and CombMNZ achieved the best Recall@100. BordaFuse performed comparably to RRF in MAP, but slightly lower in NDCG and Recall.

RRF's strong performance is likely due to its robustness in aggregating ranks without assuming consistent score distributions across systems. Unlike CombSUM and CombMNZ, which rely directly on score magnitudes, RRF is rank-based and less sensitive to normalization issues.

Overall, fusion methods significantly outperformed individual models. For example, RRF achieved an NDCG@10 of 0.7092 compared to TinyBERT's 0.6919 and MiniLM's 0.6703. This suggests that model ensembling helps combine the strengths of different cross-encoders.

4 Task 3: Analysis of Two-Model Ensemble Combinations

4.1 Setup

Based on the results of Task 2, we selected Reciprocal Rank Fusion (RRF) as the most effective ensemble method. In this task, we applied RRF to all possible two-way combinations of the three fine-tuned models: MiniLM, TinyBERT, and DistilRoBERTa.

The goal was to evaluate how well subsets of the ensemble perform in comparison to the full ensemble of all three models.

4.2 Results

Model Combination	NDCG@10	Recall@100	MAP@1000
MiniLM + DistilRoBERTa	**0.7032**	0.4985	0.4505
MiniLM + TinyBERT	0.7016	**0.5065**	0.4556
TinyBERT + DistilRoBERTa	0.7004	0.5013	**0.4600**
TinyBERT (individual)	0.6919	0.5035	0.4536
MiniLM (individual)	0.6703	0.4925	0.4288
DistilRoBERTa (individual)	0.6563	0.4727	0.4209

Table 3: Evaluation results of all two-model combinations using RRF.

4.3 Discussion

Among the three combinations, the pair **MiniLM + DistilRoBERTa** achieved the highest NDCG@10 score of 0.7032. Interestingly, this outperformed both of the individual models involved—by 4.91% over MiniLM alone and 7.15% over DistilRoBERTa alone. However, it still fell slightly behind the full three-model ensemble, which scored 0.7092.

On the other hand, the combination **TinyBERT + DistilRoBERTa** yielded the lowest performance among the pairwise ensembles (NDCG@10 = 0.7004), although still outperforming any single model.

These results suggest that while two-model ensembles can significantly improve over individual models, the inclusion of a third diverse model provides marginal yet valuable gains in robustness and ranking accuracy. It also demonstrates the strength of RRF in fusing complementary retrieval behaviors.

5 Task 4 and 5: Query Expansion with LLM - PRF based - Prompt Strategies

5.1 Prompt strategies

We performed a similar prompting strategies as the paper did. As a model have used **"HuggingFaceH4/zephyr-7b-beta"** and we used **"cross-encoder/ms-marco-MiniLM-L-6-v2"** as the Cross Encoder which is a BERT encoder.
Below we summarize the five different LLM prompt templates we evaluated. We tried to conduct extra prompting study that is similar to one in the paper such as Q2E and Q2E/PRF.

CoT
```
Answer the following query:
{query}
Give the rationale before answering.
```

CoT_distilled
```
Reformulate the query concisely.
Query: {query}
```

```
Answer:
```

Q2E
```
Write a list of keywords for the given query:
Query: {query}
Keywords:
```

Q2E/PRF
```
Write a list of keywords based on context:
Context:
{docs}
Query: {query}
Keywords:
```

CoT/PRF
```
CoT/PRF
Answer the following query based on the context:
Context: {PRF doc 1}
{PRF doc 2}
{PRF doc 3}
Query: {query}
Give the rationale before answering.
```

5.2 Evaluation Results

Model Combination	NDCG@10	Recall@100	MAP@1000
Before Expansion	**0.74**	0.53	0.5
CoT Expansion	0.67	0.49	0.43
CoT Distilled	0.71	0.53	0.49
CoT+PRF Expansion	0.61	0.44	0.36
Q2E	0.66	0.47	0.42
Q2E/PRF	0.32	0.32	0.24

Table 4: Evaluation results of Before and After Query Expansion

5.3 Discussion

It can be seen from the table that all of the methods performed poorly than the baseline method since the baseline provided more accurate results than each methods. In the paper, they report that results are improved with the CoT and CoT/PRF implementations. [2].

There might be several reasons of these differences. One of them is the differences in model selections. In the study, we used HuggingFaceH4/zephyr-7b-beta whereas in the paper they used their Flan-UL2 (20 B) and Flan-T5-XXL (11 B) whice are larger models than Zephyr-7b which could generate more accurate results. In addition to the the model size, in the paper they report that they have fine tuned the model for their queries. In short, there are key differences between models in terms of its size and its being fine-tuned or not.

Secondly, it might be because of the differences in first stage retrieval and ranking processes. Method in the paper is first expanding BM25 queries and then reranking with the same model. But our study, uses a BERT-style cross encoder to re-rank the same top-1000 candidates and then measures the statistics. This difference might be the other reason for the differences in model performances.

Another difference is the differences in the prompting strategies. In our studies' prompts, for example in CoT, we append the entire rationale into the query. Using the extra tokens might led our cross-encoder to overweight the irrelevant context in the queries. In contrast, it can be seen from the paper's Appendix A tha the study filters out trailing "The final answer is..." sentences and outputs mostly contains the keywords [2]. This concatenation might led paper to achieve improved results in CoT too.

References

[1] Elias Bassani, Diego Esteves, Raffaele Perego, and Fabrizio Silvestri. ranx.fuse: A python library for metasearch. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022. URL `https://dl.acm.org/doi/pdf/10.1145/3511808.3557207`.

[2] Rolf Jagerman, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bendersky. Query expansion by prompting large language models, 2023. URL `https://arxiv.org/abs/2305.03653`.